

Collection Class 1

ArrayList

The ArrayList class extends **AbstractList** and implements the List interface. ArrayList supports **dynamic arrays** that can grow as needed.

Standard Java arrays are of a fixed length. After arrays are created, they cannot grow or shrink, which means that you must know in advance how many elements an array will hold.

Array lists are created with an initial size. When this size is exceeded, the collection is automatically enlarged. When objects are removed, the array may be shrunk.

The important points about the Java ArrayList class are:

- Java ArrayList class can contain **duplicate elements**.
- Java ArrayList class maintains **insertion order**.
- Java ArrayList class is **non-synchronized**.

- Java ArrayList allows **random access** because the array works on an index basis.
- In ArrayList, manipulation is a little bit slower than the LinkedList in Java because a lot of shifting needs to occur if any element is removed from the array list.

Various Methods to Define an ArrayList:

```
ArrayList<Integer> list1 = new ArrayList<Integer>();  
// constructor to build empty ArrayList  
  
ArrayList<Boolean> list2 = new ArrayList<>();  
ArrayList<Integer> list3 = new ArrayList<>(10);  
// ArrayList of capacity 100  
  
ArrayList<Integer> arr = new ArrayList<>(Collections.nCopies(10, 0));  
// ArrayList with 10 elements all equal to 0
```

Modifying functions of ArrayList:

```
// Add in ArrayList  
list1.add(100);  
list1.add(700);  
  
// Above function will add the the elements in the end of the list  
  
// Get in ArrayList or to access any Index  
int num = list1.get(0);  
  
// Add element in between in ArrayList  
list1.add(3,0);
```

```
// Set element in ArrayList
list1.set(0,7);

// Delete/Modify in ArrayList
list1.remove(0);

// size of ArrayList
int sz = list1.size();

// Contains in ArrayList
System.out.println(list1.contains(200));
// it will check whether the element present in the array or not

// Sort ArrayList
Collections.sort(list1);
// It will sort the above List in the ascending order

// clear the elements from ArrayList
list1.clear();
// It will clear all the elements of the ArrayList
```

Stack

Java Collection framework provides a Stack class. This class is based on the basic principle of **last-in-first-out(LIFO)**.

Creating a stack:

```
// Create a Stack
Stack<Integer> stack = new Stack<Integer>();
```

Pushing elements in a stack:

```
// pushing elements on the top of Stack  
stack.push(5);  
stack.push(9);  
stack.push(19);  
stack.push(34);
```

Size of a stack

```
// size of stack  
int sz = stack.size();  
System.out.println(sz);
```

Accessing the top most element of the stack

```
// Printing the top element of the stack  
int top_element = stack.peek();
```

Deleting/ removing the top element.

```
// popping element from the Stack  
stack.pop();
```

Searching for an element.

```
int pos1 = stack.search(9);  
int pos2 = stack.search(10);
```

Queue

The Queue interface present in the java.util package and extends the Collection interface is used to hold the elements about to be processed in

FIFO(First In First Out) order. Being an interface the queue needs a concrete class for the declaration.

Creating a queue:

```
// declaration of queue  
Queue<Integer> queue = new LinkedList<>();
```

Pushing elements in a queue (tail):

```
// Adding Elements to the queue  
queue.add(5);  
queue.add(9);  
queue.add(19);  
queue.add(34);
```

Size of a queue:

```
// size of queue  
int sz = queue.size();  
System.out.println(sz);
```

Accessing the first (head) element of the queue:

```
//Printing the head element of the queue  
int head = queue.peek();  
System.out.println(head);
```

Deleting/ removing the first (head) element:

```
// removing the head element of the queue  
queue.remove();  
head = queue.peek();
```

```
System.out.println(head);
```

Priority Queue

A PriorityQueue is used when the objects are supposed to be processed based on the priority. It is known that a Queue follows the **First-In-First-Out algorithm**, but sometimes the elements of the queue need to be processed according to the priority, that's when the PriorityQueue comes into play. The PriorityQueue is based on the **priority heap**. The elements of the priority queue are ordered according to the natural ordering, or by a Comparator provided at queue construction time, depending on which constructor is used. It provides $O(\log(n))$ time for add and poll methods.

Creating a Priority queue:

```
// Declare PriorityQueue
PriorityQueue<Integer> pq = new PriorityQueue<Integer>();
```

Adding elements in the Priority queue;

```
// Adding elements in PriorityQueue
pq.add(3);
pq.add(6);
pq.add(1);
pq.add(8);
```

Size of a Priority queue:

```
int sz = pq.size();
System.out.println(sz);
```

Accessing the top element of the Priority queue:

```
// Printing top element of PriorityQueue  
int top = pq.peek();  
System.out.println(top);
```

Deleting/ removing the first (head) element:

```
// Deleting the top element of the PriorityQueue  
pq.remove();  
top = pq.peek();  
System.out.println(top);
```

Iterating over Priority queue:

```
Iterator iterator = pq.iterator();  
while (iterator.hasNext()) {  
    System.out.print(iterator.next() + " ");  
}
```

Retrieves and removes the head of this Priority queue:

```
top = pq.poll();  
System.out.println(top);  
top = pq.peak();  
System.out.println(top);
```

DeQue

Deque interface present in **java.util** package is a subtype of the queue interface. The Deque is related to the **double-ended queue** that supports the addition or removal of elements from either end of the data structure. It can either be used as a queue(**first-in-first-out/FIFO**) or as a stack(**last-in-first-out/LIFO**). Deque is the acronym for **double-ended queue**.

Creating a Deque:

```
Deque<Integer> dq = new ArrayDeque<Integer>(10);
```

Adding elements in the Deque:

```
// Adding element at the start of Deque  
dq.addFirst(32);  
dq.addFirst(56);  
// Adding element at the end of Deque  
dq.addLast(98);  
dq.addLast(12);
```

Size of a Deque:

```
int sz = dq.size();  
System.out.println("size of deque: "+sz);
```

Accessing the elements of the Deque:

```
// Printing first element in Deque
```



```
System.out.println("Head Element in deque: "+ dq.getFirst());  
  
// Printing last element in Deque  
System.out.println("Last Element: "+ dq.getLast());
```

Deleting/ removing the first (head) element:

```
// deleting the first element in Deque  
dq.remove();
```

Iterating over Deque:

```
// Printing elements inside the Deque from start to end  
System.out.println("Printing elements from start to end");  
for (Iterator itr = dq.iterator();  
     itr.hasNext();) {  
    System.out.println(itr.next());  
}  
  
// Printing elements inside the Deque from end to start  
System.out.println("Printing elements from end to start");  
for (Iterator dItr = dq.descendingIterator();  
     dItr.hasNext();) {  
    System.out.println(dItr.next());  
}
```

Retrieves and removes the specific element in the deque:

```
// deleting a specific element in Deque  
dq.remove(98);  
  
// deleting the last element in Deque  
dq.removeLast();
```

Next Greater Element

Problem Statement:

Given an array, print the **Next Greater Element** (NGE) for every element. The Next greater Element for an element **x**, is the first greater element on right side of **x** in the array. Elements for which no greater element exist, consider next greater element as **-1**.

Approach:

To solve this problem in **O(n)** we can use Stack in the following way.

- Push the first element to stack.
- Pick rest of the elements one by one and follow the following steps in loop.
 - Mark the current element as next.
 - If stack is not empty, compare top element of stack with next.
 - If next is greater than the top element, Pop element from stack. next is the next greater element for the popped element.
 - Keep popping from the stack while the popped element is smaller than next. next becomes the next greater element for all such popped elements.
- Finally, push the next in the stack.
- After the loop in step 2 is over, pop all the elements from the stack and print -1 as the next element for them.

Code:

```
import java.io.*;
import java.util.*;

class CodingNinja {
```

```
public static void main(String[] args){
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    ArrayList<Integer> nums = new ArrayList<>();
    ArrayList<Integer> ans = new ArrayList<>(Collections.nCopies(n,
0));

    for(int i=0;i<n;i++){
        nums.add(sc.nextInt());
    }

    // declaring a stack
    Stack<Integer> stack = new Stack<Integer>();

    for(int i=n-1;i>=0;i--){
        if(stack.isEmpty()){
            ans.set(i,-1);
            stack.push(nums.get(i));
        }
        else{
            while(!stack.isEmpty() && stack.peek()<=nums.get(i))
stack.pop();

            if(stack.isEmpty()){
                ans.set(i,-1);
                stack.push(nums.get(i));
            }
            else{
                ans.set(i,stack.peek());
                stack.push(nums.get(i));
            }
        }
    }
    System.out.println(ans);
}
```

Time Complexity: $O(n)$ where n is the number of elements.

We are traversing the array once and pushing elements in the stack and after that we keep popping them until we find the next greater in that manner. If we take the worst case we have to travel the entire array only twice. Hence the time complexity is $O(2*n) \rightarrow O(n)$.

Space Complexity : $O(n)$ where n is the number of elements.

We are taking a stack and pushing elements into it. In the worst case we have to push all the elements of the array in it therefore the space complexity is $O(n)$.

HashSet

Java **HashSet** class is used to create a collection that uses a **hash table** for storage. It inherits the **AbstractSet class** and implements **Set interface**.

HashSet contains unique elements only. HashSet allows null value. Java **LinkedHashSet** class is a Hashtable and Linked list implementation of the Set interface. It inherits the HashSet class and implements the Set interface. Java LinkedHashSet class **maintains insertion order**.

Creating a HashSet:

```
HashSet<String> set=new HashSet<>();
```

Inserting elements in HashSet

```
set.add("One");  
set.add("Two");  
set.add("Three");  
set.add("Four");
```

Size of a HashSet:

```
// size of HashSet  
int sz = set.size();
```

Deleting/ removing the Specific element:

```
// remove specific element from HashSet  
set.remove("Four");  
Iterator<String> j=set.iterator();  
while(j.hasNext()) System.out.print(j.next()+" ");  
System.out.println();
```

Iterating over Hashset:

```
// iterate over HashSet  
Iterator<String> i=set.iterator();  
while(i.hasNext()) System.out.print(i.next()+" ");  
System.out.println();
```

TreeSet

TreeSet is one of the most important implementations of the SortedSet interface in Java that uses a Tree for storage. **TreeSet** implements the SortedSet interface. So, duplicate values are not allowed. **TreeSet** is basically an implementation of a self-balancing binary search tree like a **Red-Black Tree**. Therefore operations like add, remove, and search takes $O(\log(N))$ time.

Creating a TreeSet:

```
TreeSet<String> ts = new TreeSet<>();
```

Inserting elements in TreeSet

```
ts.add("One");  
ts.add("Two");  
ts.add("Three");  
ts.add("Four");
```

Check availability:

```
// check is an element is present in TreeSet  
if(ts.contains("Five")) System.out.println("Yes");  
else System.out.println("No");
```

Accessing first and last element:

```
System.out.println("First Value " + ts.first());
System.out.println("Last Value " + ts.last());

// values just greater and smaller than the given string
System.out.println("Higher " + ts.higher("One"));
System.out.println("Lower " + ts.lower("One"));
```

Size of a TreeSet:

```
// size of treeSet
int sz = ts.size();
```

Deleting/ removing the Specific element:

```
// Deleting any specific element in TreeSet & also first and last element
from the TreeSet
ts.remove("One");
ts.pollFirst();
ts.pollLast();
```

Using comparator function:

```
class cmp implements Comparator<String> {
    public int compare(String str1, String str2){
        return str2.compareTo(str1);
    }
}
```

```
}
```

```
TreeSet<String> tree_set1 = new TreeSet<String>(new cmp());  
    tree_set1.add("A");  
    tree_set1.add("B");  
    tree_set1.add("C");  
    tree_set1.add("D");  
    tree_set1.add("E");
```

K largest Element

Problem Statement:

You are given an integer **k** and an array of integers that contain numbers in random order. Write a program to find **k largest numbers** from a given array. You need to save them in an array and return it.

Approach:

To solve this problem we can use priority queue in the following way:

- Create a priority queue **P** and insert the root node of the **max-heap** into P.
- Repeat these steps **k - 1** times:
 - Pop the greatest element from P.
 - Insert left and right child elements of the popped element. (if they exist).
- The greatest element in P is the k greatest element of the max-heap.

The initial size of the priority queue is one, and it increases by at most one at each of the **k - 1** steps. Therefore, there are maximum **k** elements in the priority queue



and the time complexity of the pop and insert operations is $O(\log k)$. Thus the total time complexity is $O(n \cdot \log k)$.

Code:

```
import java.io.*;
import java.util.*;

class CodingNinja {

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int k = sc.nextInt();
        PriorityQueue<Integer> pq = new
            PriorityQueue<Integer>(Collections.reverseOrder());

        for(int i=0;i<n;i++){
            pq.add(sc.nextInt());
        }
        while(k>0){
            System.out.println(pq.peek());
            pq.remove();
            k--;
        }
    }
}
```

Time Complexity: $O(n \cdot \log(k))$ where n is the number of elements and k is the given integer.

Space Complexity : $O(k)$ where k is the given integer.

HashMap

It provides the basic implementation of the Map interface of Java. It stores the data in **(Key,Value) pairs**. If you try to insert the duplicate key, it will replace the element of the corresponding key. HashMap **doesn't allow duplicate keys** but allows duplicate values. Java HashMap doesn't maintain the order of keys. We can also read other methods such as **clear, clone, isEmpty**.

Creating a HashMap:

```
HashMap<String, Integer> mp = new HashMap<>();
```

Inserting elements in HashMap

```
mp.put("coding",1);  
mp.put("ninja", 2);  
mp.put("course",3);
```

Size of a HashMap:

```
int sz = mp.size();  
System.out.println("size is = "+sz);
```

Deleting/ removing the Specific element:

```
// remove key/value pair from the map  
mp.remove("coding");  
System.out.println(mp);
```

Accessing and modifying the key:

```
// get value for the given key  
int val = mp.get("ninja");  
System.out.println(val);  
// changing the value of a key  
mp.put("ninja",4);  
System.out.println(mp);
```

TreeMap

It provides the basic implementation of the Map interface of Java. It stores the data in **(Key, Value) pairs**. This proves to be an efficient way of sorting and storing the **key-value pairs**. The storing order maintained by the treemap must be consistent with equals just like any other sorted map, irrespective of the explicit comparators.

Creating a TreeMap:

```
TreeMap<Integer, String> mp= new TreeMap<Integer, String>();
```

Inserting elements in TreeMap

```
mp.put(1, "Coding");  
mp.put(2, "Ninja");  
mp.put(3, "Course");
```

Check availability:

```
if(mp.containsKey(2)) System.out.println("Key present");  
else System.out.println("Key not present");
```

Accessing the given key:

```
System.out.println(mp.get(1));
```

Size of a TreeMap:

```
// size of treeSet  
int sz = mp.size();
```

Using comparator function:

```
TreeMap<Integer, String> mp1= new TreeMap<Integer, String>((a, b) -> b-a);  
  
mp1.put(1, "Coding");  
mp1.put(2, "Ninja");  
mp1.put(3, "Course");
```

```
System.out.println(mp1);
```

2 Sum

Problem Statement:

Given an integer array Arr of size **N** and an integer target, your task is to find the indices of two elements of the array such that their sum is equal to target. Return **<-1,-1>** if no such pair exists.

Approach:

We can use hashing to find the required pair of indices in one-pass. The hash map will have the value of the elements of the array as key and the first occurring index of the element as the value corresponding to the key.

While we are iterating and inserting elements into the table, we also look back to check if (target - current element) already exists in the table. If it exists, we have found a solution and return immediately, else we keep on iterating and filling our hash table.

Algorithm :

- Create a hash table and take a variable of type pair to store the indices of the required pair.
- Iterate the array and check if target - Arr[i] exists in the hash table.

- a. If it exists, update the value of the pair if this is the first pair that we have found or if this is the lexicographically smallest pair until now.
- b. Else insert the current index value in the hash map only if no such key already exists in the hash map and continue the iteration.
- Return the pair.

Code:

```
import java.io.*;
import java.util.*;

class CodingNinja {

    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int target = sc.nextInt();

        int[] nums = new int[n];
        for(int i=0;i<n;i++) nums[i] = sc.nextInt();

        TreeMap<Integer, Integer> mp= new TreeMap<Integer, Integer>();

        for(int i=0;i<n;i++){
            if(!mp.containsKey(target-nums[i])) mp.put(nums[i],i);
            else{
                System.out.println(mp.get(target-nums[i])+" "+i);
                return;
            }
        }
        System.out.println(-1+" "+-1);
    }
}
```

Time Complexity:

$O(N)$, where 'N' is the number of elements in the array.



We traverse the array containing N elements once. Since the hash table reduces the lookup time to $O(1)$. The time complexity of the whole algorithm will be $O(1) + O(N) = O(N)$.

Space Complexity

$O(N)$, where N is the size of the array.

The hash table takes $O(N)$ space in the worst case, i.e. when all the values of the array are different.